

LECTURE 17

Midterm Review

A checkpoint across Lectures 1–16

CSC323 · Operating Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- Consolidate OS structure, the process concept, and the role of the kernel
- Compare every CPU scheduling algorithm covered so far, side by side
- Review the synchronization tools used to protect shared data, and when each applies
- Revisit the classical synchronization problems and what each tests
- Self-test your recall with a short, answer-free quiz before the exam

Lectures 1–16, Four Units

Each unit hands the next one a vocabulary it assumes fluency with — Unit 4 is exactly where deadlocks (Lecture 19 onward) pick up.

Unit 1: OS Structure & Design

What an OS does, kernel designs, system calls, user/kernel mode



Unit 2: Processes, IPC & Threads

Process states, the PCB, context switches, shared memory vs. message passing, threading models



Unit 3: CPU Scheduling

Scheduling criteria, FCFS, SJF, SRTF, Priority, Round Robin, multilevel queues



Unit 4: Process Synchronization

Critical sections, semaphores, monitors, Producer-Consumer, Readers-Writers, Dining Philosophers

OS Structure & Design

A process requests privileged operations only through system calls; the mode bit enforces the boundary between user mode and kernel mode.

Kernel Design	Organization	Trade-off
Monolithic	Nearly everything runs in one kernel address space	Fast, but one bad driver can crash the whole system
Layered	Strict layers, each built only on the one below	Easier to debug; still one address space
Microkernel	Only IPC/scheduling/memory in kernel mode; rest in user-mode servers	Crash-resilient, but every service costs a message
Hybrid	Small trusted core + selected subsystems in kernel mode	Where most real OSes (Windows, macOS, Linux) land

TIP

If an exam question says "crashes the whole system," think monolithic — a microkernel’s servers are isolated processes; a crash is contained.

Processes, PCB & IPC

- A process is tracked via its PCB: state, program counter, CPU registers, memory limits, I/O status
- A process moves through 5 states (new, ready, running, waiting, terminated) — never changes state unilaterally
- A thread is the unit the CPU executes; threads of the same process share an address space, so switching between them is far cheaper than switching processes

Shared Memory

Processes read/write a region both can see directly — fast, but every access must be synchronized by the programmer

Message Passing

Processes exchange `send()/receive()` through the kernel — slower, but synchronization comes nearly for free

Threading Models

Model	Mapping	Consequence
Many-to-One	Many user threads → one kernel thread	Cheap to create, but one blocking call freezes all; no parallelism possible
One-to-One	Each user thread → its own kernel thread	True parallelism, at the cost of a kernel context switch per thread
Many-to-Many	Many user threads → a smaller/equal kernel pool	Tries to capture cheapness and parallelism at once

NOTE

A single blocking system call from any many-to-one user thread freezes every other thread in that process — the kernel sees only one schedulable entity.

CPU Scheduling Algorithms (1 of 2)

Algorithm	Preempt?	Wait-Time Optimality	Starvation Risk	Real-World Use
FCFS	No	Poor — convoy effect	None	Simple batch queues
SJF (non-preempt.)	No	Optimal among non-preemptive	High	Batch systems, known bursts
SRTF	Yes	Optimal overall	High	Theoretical lower bound
Priority	Either	Depends on priority assignment	High (use aging)	Underlies most schedulers

CPU Scheduling Algorithms (2 of 2)

Algorithm	Preempt?	Wait-Time Optimality	Starvation Risk	Real-World Use
Round Robin	Yes	Moderate — depends on quantum	None	Time-sharing, interactive systems
Multilevel Queue	Usually	Depends on fixed queue priority	Possible (lower queues)	Separating batch vs. interactive
Multilevel Feedback Queue	Yes	Good — adapts to behavior	Low	Closest to real OS schedulers

NOTE

The time quantum is Round Robin’s entire personality: too large → degrades toward FCFS; too small → context-switch overhead dominates. Multilevel feedback queues let the system adapt the effective quantum instead of fixing one value.

The Critical-Section Problem

Mutual Exclusion

Only one process in its critical section at a time

Progress

If no process is in its critical section, one of those waiting must eventually be allowed in

Bounded Waiting

No process waits an unbounded number of turns while others repeatedly cut in line

- Semaphores generalize the lock into an integer via `wait()` and `signal()`; a binary semaphore behaves like a mutex, a counting semaphore controls a pool of identical resources
- Monitors package a lock with the shared data and the only operations allowed to touch it — the compiler enforces correct use structurally

Three Classical Synchronization Problems

Producer–Consumer

Bounded buffer; tests coordinating a count (empty/full slots) alongside mutual exclusion

Readers–Writers

Many readers may overlap safely; a writer needs total exclusivity — tests asymmetric access

Dining Philosophers

Circular resource acquisition (forks); tests exactly the cyclic waiting that causes deadlock

NOTE

Dining Philosophers is a preview, not just a puzzle: every philosopher holds one fork while waiting for another — a direct physical model of hold-and-wait and circular wait, named formally in Lecture 19.

Check Yourself Before the Exam

- Q1. Why does a microkernel improve fault isolation over a monolithic kernel, and what does it give up in exchange?
- Q2. State the three requirements a correct critical-section solution must satisfy, and explain what would go wrong if one were violated.
- Q3. Four processes have the arrival/burst times from Lecture 17's table. Compute the Gantt chart and average waiting time under non-preemptive SJF.
- Q4. Why can Round Robin's performance degrade toward FCFS if the quantum is too large, and toward excessive overhead if too small?

Check Yourself Before the Exam

- Q5. A binary semaphore mutex is initialized to 1. Three processes each execute wait(mutex) in quick succession, with no signal(mutex) in between. What is the state of each process, and the final value of mutex?
- Q6. Explain, in your own words, how the Dining Philosophers problem models hold-and-wait and circular wait at the same time.
- Q7. Distinguish a process from a thread in terms of what each owns, and explain why a context switch between two threads of the same process is cheaper.
- Q8. Why is non-preemptive SJF described as "optimal among non-preemptive algorithms" rather than simply "optimal"? What beats it, and at what cost?

These questions have no printed answers on purpose — work through each on paper before the exam.

KEY TAKEAWAYS

Ready for the Midterm

- 1 The four units build in one direction: structure and processes are what the scheduler chooses among, and scheduling is why synchronization is necessary at all
- 2 Kernel design trades speed (monolithic) against fault isolation (microkernel); most real systems are hybrids
- 3 Threads share an address space and are cheap to switch between; many-to-one threading cannot achieve parallelism no matter how many user threads exist
- 4 No scheduling algorithm wins on every criterion — know the preemption, optimality, and starvation trade-offs cold
- 5 The critical-section problem's three requirements are the yardstick every synchronization tool is judged against
- 6 Producer–Consumer, Readers–Writers, and Dining Philosophers each isolate a different failure mode — the last is exactly where deadlock, starting Lecture 19, picks up

Lecture 18 turns this review into deliberate practice: four fully worked problems.